

5 R2007 DWG FILE FORMAT ORGANIZATION

5.1 Sections and pages overview

Like the R18 format the R21 format has sections and pages. There are system sections and data sections. The system sections contain information about where the data sections and their pages are in the stream. A system section only has a single page, while a data section can have multiple pages. The page map contains information about where each data page is in the file stream. The section map has information about which pages belong to which section. The file header, which is at the beginning of the file, just after the meta data, contains the stream locations of the page map and section map.

The following table shows the section and page order in the stream:

Section/page	Size	Description
Meta data	0x80	Meta data (version info etc)
File header	0x400	File header, contains page/section map addresses, sizes, CRC's etc.
Page map 1	0x400 or more	The data page map
Page map 2	0x400 or more	A copy of the data page map
AcDb:SummaryInfo		
AcDb:Preview		
AcDb:VBAProject		
AcDb:AppInfo		
AcDb:FileDepList		
AcDb:RevHistory		
AcDb:Security		
AcDb:AcDbObjects		

AcDb:ObjFreeSpace		
AcDb:Template		
AcDb:Handles		
AcDb:Classes		
AcDb:AuxHeader		
AcDb:Header		
Section map 1		The section map.
Section map 2		A copy of the section map.

5.2 R2007 Meta Data

Address	Length	Description
0x00	6	“AC1021” version string
0x06	5	5 bytes of 0x00
0x0B	1	Maintenance release version
0x0C	1	Byte 0x00, 0x01, or 0x03
0x0D	4	Preview address (long)
0x11	1	Dwg version (Acad version that writes the file)
0x12	1	Maintenance release version (Acad maintenance version that writes the file)
0x13	2	Codepage
0x15	3	Unknown (ODA writes zeroes)
0x18	4	SecurityType (long), see R2004 meta data, the definition is the same, paragraph 4.1.
0x1C	4	Unknown long

0x20	4	Summary info Address in stream
0x24	4	VBA Project Addr (0 if not present)
0x28	4	0x00000080
0x2C	4	Application Info Addr

At offset 0x80 there is a 0x400 byte section. The last 0x28 bytes of this section consists of check data, containing 5 Int64 values representing CRC's and related numbers (starting from 0x3D8 until the end). The first 0x3D8 bytes should be decoded using Reed-Solomon (255, 239) decoding, with a factor of 3. The format of this decoded data is:

Address	Length	Description
0x00	8	CRC
0x08	8	Unknown key
0x10	8	Compressed Data CRC
0x18	4	ComprLen
0x1C	4	Length2
0x20	ComprLen	Compressed Data

Note that if ComprLen is negative, then Data is not compressed (and data length is ComprLen). If ComprLen is positive, the ComprLen bytes of data are compressed, and should be decompressed using the `OdDwgR21Compressor::decompress()` function, where the decompressed size is a fixed 0x110. The decompressed data is in the following format:

Address	Length	Description
0x00	8	Header size (normally 0x70)
0x08	8	File size
0x10	8	PagesMapCrcCompressed
0x18	8	PagesMapCorrectionFactor
0x20	8	PagesMapCrcSeed

0x28	8	Pages map2offset (relative to data page map 1, add 0x480 to get stream position)
0x30	8	Pages map2Id
0x38	8	PagesMapOffset (relative to data page map 1, add 0x480 to get stream position)
0x40	8	PagesMapId
0x48	8	Header2offset (relative to page map 1 address, add 0x480 to get stream position)
0x50	8	PagesMapSizeCompressed
0x58	8	PagesMapSizeUncompressed
0x60	8	PagesAmount
0x68	8	PagesMaxId
0x70	8	Unknown (normally 0x20)
0x78	8	Unknown (normally 0x40)
0x80	8	PagesMapCrcUncompressed
0x88	8	Unknown (normally 0xf800)
0x90	8	Unknown (normally 4)
0x98	8	Unknown (normally 1)
0xA0	8	SectionsAmount (number of sections + 1)
0xA8	8	SectionsMapCrcUncompressed
0xB0	8	SectionsMapSizeCompressed
0xB8	8	SectionsMap2Id
0xC0	8	SectionsMapId
0xC8	8	SectionsMapSizeUncompressed
0xD0	8	SectionsMapCrcCompressed
0xD8	8	SectionsMapCorrectionFactor

0xE0	8	SectionsMapCrcSeed
0xE8	8	StreamVersion (normally 0x60100)
0xF0	8	CrcSeed
0xF8	8	CrcSeedEncoded
0x100	8	RandomSeed
0x108	8	Header CRC64

This section will be referred to as the **File Header** throughout the remainder of this document.

The page map is stored in a single system section page. The page size of this system section page depends on how much data is stored in it. One page should be able to fit $((\text{dataSectionPageCount} + 5) * 16)$ bytes. PagesMapOffset indicates the starting address of the Page Map section of the file, PagesMapSizeCompressed is the compressed size of this section, PagesMapSizeUncompressed is the uncompressed size, PagesMapCorrectionFactor is the correction factor used, and PagesMapCrcCompressed and PagesMapCrcUncompressed are the compressed and uncompressed CRC values, respectively. The data at PagesMapOffset is in the following format (to be referred to as “System Page” format throughout the remainder of this document) should be decoded and optionally decompressed using the `OdDwgR21FileController::loadSysPage` function. The resulting pages map data consists of a sequence of pairs, where each pair consists of an Int64 **SIZE** value, and an Int64 **ID** value. This sequence creates a set of pages where each. These values create a pages map using the following algorithm:

```
OdInt64 offset = 0;
```

```
while (!pStream->isEof())
{
    size = OdPlatformStreamer::rdInt64(*pStream);
    id = OdPlatformStreamer::rdInt64(*pStream);
    ind = id > 0 ? id : -id;

    m_pages[ind].m_id = id;
    m_pages[ind].m_size = size;
    m_pages[ind].m_offset = offset;
    offset += size;
}
```

The **File Header** value PagesMaxId indicates the largest index that will be used for the `m_pages` array.

Next, the Section Map should be loaded. The offset of the section map data is the `m_offset` value of the page with index SectionsMapId in the Page Map of the file. The File Header values SectionsMapSizeCompressed, SectionsMapSizeUncompressed, SectionsMapCrcCompressed, SectionsMapCrcUncompressed, and SectionsMapCorrectionFactor make of the remainder of the arguments to pass to the `OdDwgR21FileController::loadSysPage` function (see paragraph 5.3) for

decoding and decompression of the Section Map data. The decoded and decompressed Section Map data consists of the following attributes for each section in the file:

Address	Length	Description
0x00	8	Data size
0x08	8	Max size
0x10	8	Encryption
0x18	8	HashCode
0x20	8	SectionNameLength
0x28	8	Unknown
0x30	8	Encoding
0x38	8	NumPages. This is the number of pages present in the file for the section, but this does not include pages that contain zeroes only. A page that contains zeroes only is not written to file. If a page's data offset is smaller than the sum of the decompressed size of all previous pages, then it is to be preceded by a zero page with a size that is equal to the difference between these two numbers.
0x40	SectionNameLength x 2 [+ 2]	Unicode Section Name (2 bytes per character, followed by 2 zero bytes if name length > 0)
Repeat NumPages times:		
	8	Page data offset. If a page's data offset is smaller than the sum of the decompressed size of all previous pages, then it is to be preceded by a zero page with a size that is equal to the difference between these two numbers.
	8	Page Size
	8	Page ID
	8	Page Uncompressed Size
	8	Page Compressed Size

	8	Page Checksum
	8	Page CRC

This data repeats until the decoded & decompressed Section Map data is exhausted, giving a set of Sections, where each section can contain data for an arbitrary number of pages. The data from all pages together forms a section in the file. Each page may be optionally RS encoded, compressed, or encrypted. The `OdR21PagedStream` class implements RS decoding, decompression, and decryption of the page data within a section (see `OdDwgR21FileSection::read()` for sample code to set up an `OdR21PagedStream` object).

The section map may contain the following sections (in this order, the order in the file stream is different):

Section Name	Description	Property	Value
AcDb:Security	Contains information regarding password and data encryption. This section is optional.	hashcode	0x4a0204ea
		pagesize	0xf800
		encryption	0
		encoding	1
AcDb:FileDepList	Contains file dependencies (e.g. IMAGE files, or fonts used by STYLE).	hashcode	0x6c4205ca
		pagesize	If no entries, 0x100, otherwise 0x80 * (countEntries + (countEntries >> 1))
		encryption	2
		encoding	1
AcDb:VBAProject	Contains VBA Project data for this drawing (optional section)	hashcode	0x586e0544
		pagesize	VBA data size + 0x80 rounded to the next 0x20.
		encryption	2
		encoding	1
AcDb:AppInfo	Contains information about the application	hashcode	0x3fa0043e

	that wrote the .dwg file (encrypted = 2).	pagesize	0x300
		encryption	0
		encoding	1
AcDb:Preview	Bitmap preview for this drawing.	hashcode	0x40aa0473
		pagesize	Default 0x400, if image is written, preview size rounded to the next 0x20 bytes.
		encryption	1 if properties are encrypted, 0 otherwise
		encoding	1
AcDb:SummaryInfo	Contains fields like Title, Subject, Author.	hashcode	0x717a060f
		pagesize	0x80
		encryption	1 if properties are encrypted, 0 otherwise
		encoding	1
AcDb:RevHistory	Revision history	hashcode	0x60a205b3
		pagesize	0x1000
		encryption	0
		encoding	4
		compressed	true
AcDb:AcDbObjects	Database objects	hashcode	0x674c05a9
		pagesize	0xf800
		encryption	1 if data is encrypted, 0 otherwise

		encoding	4
		compressed	true
AcDb:ObjFreeSpace		hashcode	0x77e2061f
		pagesize	0xf800
		encryption	0
		encoding	4
		compressed	true
AcDb:Template	Template (Contains the MEASUREMENT system variable only.)	hashcode	0x4a1404ce
		pagesize	0x400
		encryption	0
		encoding	4
		compressed	true
AcDb:Handles	Handle list with offsets into the AcDb:AcDbObjects section	hashcode	0x3f6e0450
		pagesize	0xf800
		encryption	1 if data is encrypted, 0 otherwise
		encoding	4
		compressed	true
AcDb:Classes	Custom classes section	hashcode	0x3f54045f
		pagesize	0xf800
		encryption	1 if data is encrypted, 0 otherwise
		encoding	4
		compressed	true

AcDb:AuxHeader		hashcode	0x54f0050a
		pagesize	0x800
		encryption	0
		encoding	4
		compressed	true
AcDb:Header	Contains drawing header variables	hashcode	0x32b803d9
		pagesize	0x800
		encryption	1 if data is encrypted, 0 otherwise
		encoding	4
		compressed	True
AcDb:Signature	Not written by ODA		

By default data/properties are not encrypted. Encryption still needs to be described.

5.2.1 File header creation

Creating the R21 file header is very complex:

Compute and set all the file header fields. In this process also compute CRC's and generate check data, derived from a CRC seed value (paragraph 5.2.1.1).

Write the file header data to a buffer and calculate/write the 64-bit CRC (paragraph 5.2.1.2).

Compress the file header data and calculate the 64-bit CRC (paragraph 5.2.1.3).

Create a checking sequence and calculate a CRC over this sequence data (paragraph 5.2.1.4).

Create a buffer in preparation of Reed-Solomon encoding (Pre-Reed-Solomon encoded data). This contains checking sequence, compressed CRC, compressed size, compressed data and random data (as padding) (paragraph 5.2.1.5).

Encode the data using Reed-Solomon (for error correction).

Write the encoded data, followed by the check data from the first step.

5.2.1.1 Calculating the file header CRC's and check data

The file header data consists of regular data fields and CRC values and check data to verify the data's correctness. All fields pertaining to the file header's correctness are discussed in more detail in the following paragraphs. Note that the order of CRC calculation is important, so the order of the following paragraphs should be used.

5.2.1.1.1 RandomSeed

Is filled with the CRC random encoding's seed (see paragraph 5.11).

5.2.1.1.2 CrcSeed

The ODA always initializes this with value 0.

5.2.1.1.3 SectionsMapCrcSeed

Is filled with crcSeed initially. Then it's encoded using the CRC random encoding as described in paragraph 5.11.

5.2.1.1.4 PagesMapCrcSeed

Is filled with crcSeed initially. Then it's encoded using the CRC random encoding as described in paragraph 5.11.

5.2.1.1.5 Check data

The check data for the file header page is present at the end of the header page at location 0x3d8. It contains data generated based on the CrcSeed and the current state of the CRC random encoder. The check data contains the following UInt64 fields (computed in this order):

Random value 1 (third value in stream)

Random value 2 (fourth value in stream)

Encoded CRC Seed (fifth value in stream)

Normal 64-bit CRC (first value in stream)

Mirrored 64-bit CRC (second value in stream)

Random value 1 is set to the CRC random encoder's next random value.

Random value 2 is set to the CRC random encoder's next random value.

The Encoded CRC seed is gotten by letting the CRC random encoder encode the CRC seed.

The normal 64-bit CRC value is calculated as follows. A buffer of 8 UInt64 values is created and initialized with zeroes. The values are encoded using the Encode function below:

```
UInt64 Encode(UInt64 value, UInt64 control) {
    Int32 shift = (Int32)(control & 0x1f);
    if (shift != 0) {
        value = (value << shift) | (value >> (64 - shift));
    }
    return value;
}
```

The buffer is initialized by encoding several values. Later this buffer becomes the input to a normal 64-bit CRC calculation:

```
UInt64 CalculateNormalCrc() {
    UInt64[] buffer = new UInt64[8];
    buffer[0] = Encode(random1, random2);
    buffer[1] = Encode(buffer[0], buffer[0]);
    buffer[2] = Encode(random2, buffer[1]);
    buffer[3] = Encode(buffer[2], buffer[2]);
    buffer[4] = Encode(random1, buffer[3]);
    buffer[5] = Encode(buffer[4], buffer[4]);
    buffer[6] = Encode(buffer[5], buffer[5]);
    buffer[7] = Encode(buffer[6], buffer[6]);

    // Convert each UInt64 in the buffer from big-endian to little-endian if
    // the machine is big-endian.
    ...

    UInt64 normalCrc = CalculateNormalCrc64(buffer, 64, ~random2);
    return normalCrc;
}
```

Similarly the mirrored CRC value is calculated:

```
UInt64 CalculateMirroredCrc() {
    UInt64[] buffer = new UInt64[8];
    buffer[0] = Encode(random1, random2);
    buffer[1] = Encode(normalCrc, buffer[0]);
    buffer[2] = Encode(random2, buffer[1]);
    buffer[3] = Encode(normalCrc, buffer[2]);
```

```

    buffer[4] = Encode(random1, buffer[3]);
    buffer[5] = Encode(normalCrc, buffer[4]);
    buffer[6] = Encode(random2, buffer[5]);
    buffer[7] = Encode(buffer[6], buffer[6]);

    // Convert each UInt64 in the buffer from big-endian to little-endian if
    // the machine is big-endian.
    ...

    UInt64 mirroredCrc = CalculateMirroredCrc64(buffer, 64, ~random1);
    return mirroredCrc;
}

```

5.2.1.1.6 CrcSeedEncoded

Encoded value of CrcSeed, using the CRC random encoding as described in paragraph 5.11.

5.2.1.2 Calculate file header data 64-bit CRC (decompressed)

The last field in the file header is a normal 64-bit CRC (see paragraph 5.12) which is the CRC calculated from the file header data, including the 64-bit CRC with value zero. The CRC seed value is 0, and then updated with method UpdateSeed2 before calling UpdateCrc (see again paragraph 5.12). The initial CRC value of 0 is replaced with the calculated value.

5.2.1.3 Compress and calculate 64-bit CRC (compressed)

The file header data is compressed. If the compressed data is not shorter than the uncompressed data, then the uncompressed data itself is used. Another normal 64-bit CRC value is calculated from the resulting data (see paragraph 5.12).

5.2.1.4 Create checking sequence and 64-bit CRC

Another checking sequence of 2 UInt64 values is created, very similar to the check data in paragraph 5.2.1.1.5. The first value is filled with the next value from the random encoder (see paragraph 5.11). The second value is calculated using the check data's Encode function, with the first sequence value passed as first (value) and second (control) parameter. The sequence bytes are then converted to little endian format. The last step is calculating a normal 64-bit CRC value (see paragraph 5.12). The CRC seed value is 0, updated by method UpdateSeed1.

5.2.1.5 Create a buffer in preparation of Reed-Solomon encoding

In preparation of the next step, which is Reed-Solomon (RS) encoding, a buffer is created which is going to be encoded. The size of this buffer is 3 x 239 bytes (239 is the RS data size for a block (k) used for system pages, see paragraph 5.13). First a block is created, of which the size is a multiple of 8 bytes:

Position	Size	Description
0	8	Checking sequence CRC (paragraph 5.2.1.4)
8	8	Checking sequence first UInt64 value (paragraph 5.2.1.4)
16	8	Compressed data CRC (paragraph 5.2.1.3)
24	8	Compressed data size. In case the compressed data size is larger than the uncompressed data size, then the negated uncompressed data size is written.
32	n	Compressed data in case the size is smaller than the uncompressed data size. Otherwise the uncompressed data.
32 + n	m	Padding so the block size is a multiple of 8 bytes. The padding bytes are gotten from the CRC random encoding, see paragraph 5.11.

This block is repeated as many times as possible within the buffer. The remaining bytes are filled using random padding data from the CRC random encoding (see paragraph 5.11).

5.2.1.6 Encode the data using Reed-Solomon

In this step the header data is encoded using the Reed-Solomon (RS) encoding for interleaved system pages (see paragraph 5.13). The encoded size is 3 x 255 bytes. The remaining bytes of the page (of total size 0x400) are filled using random padding data from the CRC random encoding (see paragraph 5.11).

5.2.1.7 Add check data at the end of the page

The last 0x20 bytes of the page should be overwritten using the check data, calculated in paragraph 5.2.1.1.5. The page size remains 0x400 bytes.

5.2.1.8 Write the file header to the file stream

The file header is written to position 0x80 and to the end of the file stream.

5.3 System section page

The system section page is used by the data section map and the section page map.

Inputs for writing a system section page are:

- The data.
- The 64-bit CRC seed.

- The page size (minimum 0x400). The page size is determined from the decompressed data size as described in paragraph 5.3.1.

Outputs are:

- Compressed and Reed-Solomon (RS) encoded data.
- Derived properties of the (compressed/encoded) data: compressed 64-bit CRC, decompressed 64-bit CRC, data repeat count (or data factor). These derived properties are written in the file header (see paragraph 5.2).

First the 64-bit CRC of the decompressed data is calculated, using the mirrored 64-bit CRC calculation (see paragraph 5.12). This uses the `UpdateSeed1` method to update the CRC seed before entering the CRC computation.

Next step is compression. If the compressed data isn't shorter than the original data, then the original data is used instead of the compressed data.

Of the resulting data (either compressed or not), another 64-bit CRC is computed (similarly to described above).

The resulting data is padded with zeroes so the length is a multiple of the CRC block size (8).

Now the resulting data is repeated as many times as possible within the page, RS encoded (see paragraph 5.13) and padded. The maximum RS block count (integer) is the page size divided by the RS codeword size (255). The maximum RS pre-encoded size is the maximum RS block count times the k-value of the RS system page encoding (239). So the data repeat count is the maximum RS pre-encoded size divided by the resulting (padded) data length. Next a buffer is created, with the resulting (padded) data repeated (data repeat count times). This buffer is encoded using RS encoding for system pages, interleaved. Note that the actual RS block count is less than or equal to the maximum RS block count calculated above. The encoded size is the RS block count times 255. The final step is to add padding using random data from the random encoding to fill the remainder of the page, see paragraph 5.11.

5.3.1 System section page size calculation

The data stored in a system section is first padded until its size is a multiple of the CRC block size (8). This is called the aligned size. The Reed-Solomon encoded aligned data should fit the system section at least two times. The minimum page size is 0x400 bytes.

The system section page size can be calculated from the uncompressed data size in bytes as shown in the following pseudo code (function `GetSystemPageSize`):

```
const Int32 CrcBlockSize = 8;
const Int32 PageAlignSize = 0x20;
const Int32 ReedSolomonDataBlockSize = 239;
const Int32 ReedSolomonCodewordSize = 255;
```

```

public static UInt64 GetAlignedPageSize(UInt64 pageSize) {
    UInt64 result = (UInt64)((Int64)(pageSize + PageAlignSize - 1) & (Int64)(-PageAlignSize));
    return result;
}

public static UInt64 GetSystemPageSize(UInt64 dataSize) {
    UInt64 alignedSize = (UInt64)((Int64)(dataSize + CrcBlockSize - 1) & (Int64)(-CrcBlockSize));
    // The page should fit the data at least 2 times.
    UInt64 filePageSize = ((alignedSize * 2) + ReedSolomonDataBlockSize - 1) /
        ReedSolomonDataBlockSize * ReedSolomonCodewordSize;
    if (filePageSize < 0x400) {
        filePageSize = 0x400;
    } else {
        filePageSize = GetAlignedPageSize(filePageSize);
    }
    return filePageSize;
}

```

5.4 Data section page

Data sections are used for all sections except the data section map and the section page map. The section's data is partitioned into pages, each of Max size length, except for the last page which may be of size less than Max size. The following steps are taken when writing data page.

First a 32-bit data checksum of the page's data is calculated. The pseudo code for this calculation is presented in paragraph 5.4.1.

Next the page data is optionally compressed (depending on the section). If the compressed data isn't shorter than the original data, then this page's data is not compressed.

If the file is encrypted, the page is encrypted (to be described).

The page's 64-bit CRC is calculated (mirrored CRC, see paragraph 5.12). The page CRC seed is the file's CRC seed updated using UpdateSeed1 (see again paragraph 5.12).

Pad the data with zero bytes so the size becomes a multiple of the CRC block size (0x8).

The data is Reed-Solomon encoded (see paragraph 5.13). Depending on the section encoding, the data is either interleaved (value 4) or not (value 1).

The page start position should be aligned on a 0x20 byte boundary (if all is well nothing has to be done at this point to achieve this). The data is written and padded with zero bytes so the stream position is again at a 0x20 byte boundary.

Finally the current page ID is incremented.

5.4.1 Data section page checksum

The function below shows how to calculate the 32-bit data page checksum:

```

UInt32 GetChecksum(UInt64 seed, byte[] data, UInt32 start, UInt32 length) {
    seed = (seed + length) * 0x343fd + 0x269ec3;
    UInt32 sum1 = (UInt32) (seed & 0xffff);
    UInt32 sum2 = (UInt32) ((seed >> 0x10) & 0xffff);

    fixed (byte* dataStartPtr = data) {
        byte* dataPtr = dataStartPtr + start;
        while (length != 0) {
            UInt32 bigChunkLength = System.Math.Min(0x15b0, length);
            length -= bigChunkLength;

            // Process small chunks of 8 bytes each.
            UInt32 smallChunkCount = bigChunkLength >> 3;
            while (smallChunkCount-- > 0) {
                UpdateSums2Bytes(dataPtr + 6, sum1, sum2);
                UpdateSums2Bytes(dataPtr + 4, sum1, sum2);
                UpdateSums2Bytes(dataPtr + 2, sum1, sum2);
                UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
                dataPtr += 8;
            }

            // Processing remaining 0..7 bytes.
            UInt32 smallChunkRemaining = bigChunkLength & 7;
            if (smallChunkRemaining > 0) {
                switch (smallChunkRemaining) {
                    case 1:
                        UpdateSums1Byte(dataPtr + 0, sum1, sum2);
                        break;
                    case 2:
                        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
                        break;
                    case 3:
                        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
                        UpdateSums1Byte(dataPtr + 2, sum1, sum2);
                        break;
                    case 4:
                        UpdateSums2Bytes(dataPtr + 2, sum1, sum2);
                        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
                        break;
                    case 5:
                        UpdateSums2Bytes(dataPtr + 2, sum1, sum2);

```

```

        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
        UpdateSums1Byte(dataPtr + 4, sum1, sum2);
        break;
    case 6:
        UpdateSums2Bytes(dataPtr + 2, sum1, sum2);
        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
        UpdateSums2Bytes(dataPtr + 4, sum1, sum2);
        break;
    case 7:
        UpdateSums2Bytes(dataPtr + 2, sum1, sum2);
        UpdateSums2Bytes(dataPtr + 0, sum1, sum2);
        UpdateSums2Bytes(dataPtr + 4, sum1, sum2);
        UpdateSums1Byte(dataPtr + 6, sum1, sum2);
        break;
    }
    dataPtr += smallChunkRemaining;
}

sum1 %= 0xffff1;
sum2 %= 0xffff1;
}

return (sum2 << 0x10) | (sum1 & 0xffff);
}

private static unsafe void UpdateSums1Byte(byte* p, UInt32& sum1, UInt32& sum2) {
    sum1 += *p;
    sum2 += sum1;
}

private static unsafe void UpdateSums2Bytes(byte* p, UInt32& sum1, UInt32& sum2) {
    UpdateSums1Byte(p, sum1, sum2);
    UpdateSums1Byte(p + 1, sum1, sum2);
}

```

5.5 AcDb:Security Section

The AcDb:Security section is optional in the file—it is present if the file was saved with a password. The data in this section is in the same format as in the R2004 format, 2 unknown 32-bit integers, a 32-bit integer with value 0xABCDABCD, etc.

5.6 AcDb:AuxHeader Section

This section is in the same format as in R2004. See details in chapter 27.

5.7 AcDb:Handles Section

This section is in the same format as in R2004.

5.8 AcDb:Classes Section

This section contains the defined classes for the drawing. It contains a new string stream for unicode string—see the Objects Section for a description of how to extract the string stream from an object.

```

SN : 0x8D 0xA1 0xC4 0xB8 0xC4 0xA9 0xF8 0xC5 0xC0 0xDC 0xF4 0x5F 0xE7 0xCF 0xB6 0x8A.
RL : size of class data area in bytes
RL : total size in bits
BL : Maximum class number
B : bool value
    : Class Data (format described below)
X : String stream data
B : bool value (true if string stream data is present).
```

Class data (repeating):

```

BS : classnum
BS : proxy flags:
    Erase allowed = 1,
    transform allowed = 2,
    color change allowed = 4,
    layer change allowed = 8,
    line type change allowed = 16,
    line type scale change allowed = 32,
    visibility change allowed = 64,
    cloning allowed = 128,
    Lineweight change allowed = 256,
    Plot Style Name change allowed = 512,
    Disables proxy warning dialog = 1024,
    is R13 format proxy= 32768
TU : appname
TU : cplusplusclassname
TU : classdxfname
B : wasazombie
BS : itemclassid -- 0x1F2 for classes which produce entities, 0x1F3 for classes which
    produce objects.
```

```

BL : Number of objects created of this type in the current DB (DXF 91).
BL : Dwg Version
BL : Maintenance release version.
BL : Unknown
BL : Unknown (normally 0L)

```

We read sets of these until we exhaust the data.

5.9 AcDb:Header Section

This section contains the “DWG Header Variables” data in a similar format as R15 files (see details in the DWG HEADER VARIABLES section of this document), except that string data is separated out into a string stream. See the Objects Section for details about string stream location within an object. Also, the handles are separated out into a separate stream at the end of the header, in the same manner as is done for Objects.

5.10 Decompression

The compression uses another variant of the LZ77 algorithm, different from the one used in R18. Like the R18 compression, the compressed stream (source buffer) contains opcodes, offsets and lengths of byte chunks to be copied from either compressed or decompressed buffer.

An opcode consists of a single byte. The first byte contains the first opcode. If the first opcode’s high nibble equals a 2, then:

- the source buffer pointer is advanced 2 bytes, and a *length* is read from the next byte, bitwise and-ed with 0x07
- the pointer is advanced another byte (3 bytes in total).

Next the decompression enters a loop. A byte chunk from the compressed stream is followed by one or more byte chunks from the decompressed stream. The last chunk may be a compressed chunk.

5.10.1 Copying a compressed chunk

If the *length* was zero, it is read from the source buffer. The following pseudo function reads the length:

```

UInt32 ReadLiteralLength(byte[] buffer) {
    UInt32 length = opCode + 8;
    if (length == 0x17) {
        UInt32 n = buffer[sourceIndex++];
        length += n;
        if (n == 0xff) {
            do {

```

```

        n = buffer[sourceIndex++];
        n |= (UInt32)(buffer[sourceIndex++] << 8);
        length += n;
    } while (n == 0xffff);
}
}
}

```

Next *length* bytes are copied from the source buffer and the source buffer pointer is advanced to one after the copied bytes. The order of bytes in source and target buffer are different. The copying happens in chunks of 32 bytes, and the remainder is copied using a specific copy function for each number of bytes (so 31 separate copy functions). For copying 1-32 bytes, a combination of sub byte blocks is made, according to the following table (the smallest block is 1 byte):

Byte count	Byte count [source array index]
1	1 [0]
2	1 [1], 1[0]
3	1 [2], 1[1], 1[0]
4	1 [0], 1 [1], 1 [2], 1 [3]
5	1 [4], 4 [0]
6	1 [5], 4 [1], 1 [0]
7	2 [5], 4 [1], 1 [0]
8	4 [0], 4[4]
9	1 [8], 8 [0]
10	1 [9], 8 [1], 1 [0]
11	2 [9], 8 [1], 1 [0]
12	4 [8], 8 [0]
13	1 [12], 4 [8], 8 [0]
14	1 [13], 4 [9], 8 [1], 1[0]

15	2 [13], 4 [9], 8 [1], 1[0]
16	8 [8], 8 [0]
17	8 [9], 1 [8], 8 [0]
18	1 [17], 16 [1], 1 [0]
19	3 [16], 16 [0]
20	4 [16], 16 [0]
21	1 [20], 4 [16], 16 [0]
22	2 [20], 4 [16], 16 [0]
23	3 [20], 4 [16], 16 [0]
24	8 [16], 16 [0]
25	8 [17], 1 [16], 16 [0]
26	1 [25], 8 [17], 1 [16], 16 [0]
27	2 [25], 8 [17], 1 [16], 16 [0]
28	4 [24], 8 [16], 16 [0]
29	1 [28], 4 [24], 8 [16], 16 [0]
30	2 [28], 4 [24], 8 [16], 16 [0]
31	1 [30], 4 [26], 8 [18], 16 [2], 2 [0]
32	16 [16], 16 [0]

To copy any number of bytes, first blocks of 32 bytes are copied. The remainder (1 – 31) is copied using one of the other 31 byte block copy functions as outlined in the table above.

5.10.2 Copying decompressed chunks

After copying a compressed chunk, one or more decompressed chunks are copied (unless the compressed chunk was the last chunk). First an opcode byte is read. Depending on the opcode the source buffer offset, length and next opcode are read.

```
private void ReadInstructions(
    byte[] srcBuf,
    UInt32 srcIndex,
    ref byte opCode,
    out UInt32 sourceOffset,
    out UInt32 length
) {
    switch ((opCode >> 4)) {
        case 0:
            length = (opCode & 0xf) + 0x13;
            sourceOffset = srcBuf[srcIndex++];
            opCode = srcBuf[srcIndex++];
            length = ((opCode >> 3) & 0x10) + length;
            sourceOffset = ((opCode & 0x78) << 5) + 1 + sourceOffset;
            break;

        case 1:
            length = (opCode & 0xf) + 3;
            sourceOffset = srcBuf[srcIndex++];
            opCode = srcBuf[srcIndex++];
            sourceOffset = ((opCode & 0xf8) << 5) + 1 + sourceOffset;
            break;

        case 2:
            sourceOffset = srcBuf[srcIndex++];
            sourceOffset = ((srcBuf[srcIndex++] << 8) & 0xff00) | sourceOffset;
            length = opCode & 7;
            if ((opCode & 8) == 0) {
                opCode = srcBuf[srcIndex++];
                length = (opCode & 0xf8) + length;
            } else {
                sourceOffset++;
                length = (srcBuf[srcIndex++] << 3) + length;
                opCode = srcBuf[srcIndex++];
                length = ((opCode & 0xf8) << 8) + length + 0x100;
            }
            break;

        default:
            length = opCode >> 4;
            sourceOffset = opCode & 15;
            opCode = srcBuf[srcIndex++];
            sourceOffset = (((opCode & 0xf8) << 1) + sourceOffset) + 1;
            break;
    }
}
```

```

    }
}

```

Below is the pseudocode for the decompressed chunk copy loop:

```

private UInt32 CopyDecompressedChunks(
    byte[] srcBuf,
    UInt32 srcIndex,
    UInt32 compressedEndIndexPlusOne,
    byte[] dstBuf,
    UInt32 outputIndex
) {
    UInt32 length = 0;
    byte opCode = srcBuf[inputIndex++];
    inputIndex++;
    UInt32 sourceOffset;
    ReadInstructions(srcBuf, srcIndex, ref opCode, out sourceOffset, out length);
    while (true) {
        CopyBytes(dstBuf, outputIndex, length, sourceOffset);
        outputIndex += length;
        length = opCode & 7;
        if ((length != 0) || (inputIndex >= compressedEndIndexPlusOne)) {
            break;
        }
        opCode = srcBuf[inputIndex];
        inputIndex++;
        if ((opCode >> 4) == 0) {
            break;
        }
        if ((opCode >> 4) == 15) {
            opCode &= 15;
        }
        ReadInstructions(srcBuf, srcIndex, ref opCode, out sourceOffset, out length);
    }
    return outputIndex;
}

```

5.11 CRC random encoding

Some CRC values are encoded by taking 10 bits from an UInt16 and adding bits from a pseudo random encoding table to form a UInt64 result. The decoding does the opposite, it takes an encoded UInt64, and extracts the 10 data bits from it and returns the result in an UInt16. The pseudo random encoding table holds 0x270 UInt32 values and is generated from a single UInt64 seed value. An index points to an entry into this table. As values are encoded, this index loops through this table. When the counter reaches 0x270 it is reset to 0 again.

From the encoding table a padding data table is calculated. This padding data is used to add padding bytes until the proper byte alignment is achieved in the main file stream. The byte order of the padding bytes in memory has to be little endian, so the encoding table is also stored in little endian format. Whenever retrieving data as a UInt32 from the encoding table, the original endianness of the bytes must be restored (depending on the machine the 4 bytes are thus reversed or not).

The following pseudocode shows how to generate the pseudo random encoding table and the padding table:

```
public void Init(UInt64 seed) {
    encodingTable = new UInt32[0x270];
    this.seed = seed;
    index = 0;

    encodingTable[0] = ((UInt32) seed * 0x343fd) + 0x269ec3;
    encodingTable[1] = ((UInt32) (seed >> 32) * 0x343fd) + 0x269ec3;
    UInt32 value = encodingTable[1];
    encodingTable[0] = PlatformUtil.ToLittleEndian(encodingTable[0]);
    encodingTable[1] = PlatformUtil.ToLittleEndian(encodingTable[1]);
    for (UInt32 i = 2; i < 0x270; i++) {
        value = (((value >> 0x1e) ^ value) * 0x6c078965) + i;
        encodingTable[i] = PlatformUtil.ToLittleEndian(value);
    }

    InitPadding();
}

private void InitPadding() {
    padding = new UInt32[0x80];
    for (int i = 0; i < 0x80; i++) {
        UpdateIndex();
        padding[i] = encodingTable[index];
        index++;
    }
}

private void UpdateIndex() {
    if (index >= 0x270) {
        index = 0;
    }
}
```

The encoding method takes a UInt16 argument, and encodes the 10 least significant bits into a UInt64 (spread evenly), and uses values from the encoding table to fill the remaining 54 bits. Below is the pseudocode for encoding:

```
public UInt64 Encode(UInt32 value) {
    UInt64 random = GetNextUInt64();
    UInt32 lo = (UInt32)(random & 0x0df7df7df);
    UInt32 hi = (UInt32)((random >> 32) & 0x0f7df7df7);
    if ((value & 0x200) != 0) {
        lo |= 0x20;
    }
    if ((value & 0x100) != 0) {
        lo |= 0x800;
    }
    if ((value & 0x80) != 0) {
        lo |= 0x20000;
    }
    if ((value & 0x40) != 0) {
        lo |= 0x800000;
    }
    if ((value & 0x20) != 0) {
        lo |= 0x20000000;
    }

    if ((value & 0x10) != 0) {
        hi |= 0x08;
    }
    if ((value & 0x8) != 0) {
        hi |= 0x200;
    }
    if ((value & 0x4) != 0) {
        hi |= 0x8000;
    }
    if ((value & 0x2) != 0) {
        hi |= 0x200000;
    }
    if ((value & 0x1) != 0) {
        hi |= 0x8000000;
    }
    return lo | ((UInt64)hi << 32);
}

public UInt64 GetNextUInt64() {
    index += 2;
    UpdateIndex();

    UInt32 low = PlatformUtil.FromLittleEndian(encodingTable[index]);
    UInt32 hi = PlatformUtil.FromLittleEndian(encodingTable[index + 1]);
}
```

```
    UInt64 result = low | (hi << 32);  
    return result;  
}
```

The decoding does the opposite: it takes an encoded UInt64 and extracts the 10 data bits from it:

```
public UInt32 Decode(UInt64 value) {  
    UInt32 result = 0;  
  
    UInt32 hi = (UInt32)(value >> 32);  
    if ((hi & 0x80000000) != 0) {  
        result |= 0x01;  
    }  
    if ((hi & 0x2000000) != 0) {  
        result |= 0x02;  
    }  
    if ((hi & 0x80000) != 0) {  
        result |= 0x04;  
    }  
    if ((hi & 0x200) != 0) {  
        result |= 0x08;  
    }  
    if ((hi & 0x8) != 0) {  
        result |= 0x10;  
    }  
  
    UInt32 lo = (UInt32)value;  
    if ((lo & 0x200000000) != 0) {  
        result |= 0x20;  
    }  
    if ((lo & 0x8000000) != 0) {  
        result |= 0x40;  
    }  
    if ((lo & 0x20000) != 0) {  
        result |= 0x80;  
    }  
    if ((lo & 0x800) != 0) {  
        result |= 0x100;  
    }  
    if ((lo & 0x20) != 0) {  
        result |= 0x200;  
    }  
  
    return result;  
}
```

}

5.1264-bit CRC calculation

DWG file format version 2007 uses 64-bit CRC values in the file header. The calculation uses a CRC lookup table with 256 64-bit values. There are two flavors of 64-bit CRC's:

normal (see ECMA-182: <http://www.ecma-international.org/publications/standards/Ecma-182.htm>),

mirrored, where the actual CRC computation is shifting bits the other way around.

Also the CRC tables used are different for these two versions. The way the CRC is computed for an array of bytes is the same for both CRC flavors. Only the way the CRC for a single byte is calculated is different (function CalculateCrcFor1Byte). For byte counts 1-8 the CRC calculation is ordered as follows:

Byte count	Sequence of blocks byte counts [source offset for each block]
1	1 [0]
2	1 [0], 1 [1]
3	2 [0], 1 [2]
4	2 [2], 2 [0]
5	4 [0], 1 [4]
6	4 [0], 2 [4]
7	4 [0], 3 [4]
8	4 [4], 4 [0]

For byte counts greater than 8, first blocks of 8 bytes each are processed. The remainder is processed according to the table above.

At the end of the CRC calculation the CRC value is inverted (bitwise not) for the normal CRC (not the mirrored CRC).

In addition to the CRC calculation itself there are two CRC initialization functions, called before calculating the CRC. Which one is used depends on the context.

```

UInt3264 UpdateSeed1(UInt3264 seed, UInt32 dataLength) {
    seed = (seed + dataLength) * 0x343fdUL + 0x269ec3UL;
    seed |= seed * (0x343fdUL << 32) + (0x269ec3UL << 32);
    seed = ~seed;
    return seed;
}

UInt3264 UpdateSeed2(UInt3264 seed, UInt32 dataLength) {
    seed = (seed + dataLength) * 0x343fdUL + 0x269ec3UL;
    seed = seed * ((1UL << 32) + 0x343fdUL) + (dataLength + 0x269ec3UL);
    seed = ~seed;
    return seed;
}

```

5.12.1 Normal CRC

The CRC for a single byte is calculated as follows:

```

UInt8 CalculateCrcFor1Byte(UInt8 data, UInt64 crc) {
    return crcTable[(data ^ (crc >> 56)) & 0xff] ^ (crc << 8);
}

```

The CRC table is initialized with the following values:

```

0x0000000000000000, 0x42f0e1eba9ea3693, 0x85e1c3d753d46d26, 0xc711223cfa3e5bb5,
0x493366450e42ecdf, 0x0bc387aea7a8da4c, 0xccd2a5925d9681f9, 0x8e224479f47cb76a,
0x9266cc8a1c85d9be, 0xd0962d61b56fef2d, 0x17870f5d4f51b498, 0x5577eeb6e6bb820b,
0xdb55aacf12c73561, 0x99a54b24bb2d03f2, 0x5eb4691841135847, 0x1c4488f3e8f96ed4,
0x663d78ff90e185ef, 0x24cd9914390bb37c, 0xedcbb28c335e8c9, 0xa12c5ac36adfe5a,
0x2f0e1eba9ea36930, 0x6dfeff5137495fa3, 0xaaefdd6dcd770416, 0xe81f3c86649d3285,
0xf45bb4758c645c51, 0xb6ab559e258e6ac2, 0x71ba77a2dfb03177, 0x334a9649765a07e4,
0xbd68d2308226b08e, 0xff9833db2bcc861d, 0x388911e7d1f2dda8, 0x7a79f00c7818eb3b,
0xcc7af1ff21c30bde, 0x8e8a101488293d4d, 0x499b3228721766f8, 0x0b6bd3c3dbfd506b,
0x854997ba2f81e701, 0xc7b97651866bd192, 0x00a8546d7c558a27, 0x4258b586d5bfbcb4,
0x5e1c3d753d46d260, 0x1cecdc9e94ace4f3, 0xdbdfdea26e92bf46, 0x990d1f49c77889d5,
0x172f5b3033043ebf, 0x55dfbadb9aee082c, 0x92ce98e760d05399, 0xd03e790cc93a650a,
0xaa478900b1228e31, 0xe8b768eb18c8b8a2, 0x2fa64ad7e2f6e317, 0x6d56ab3c4b1cd584,
0xe374ef45bf6062ee, 0xa1840eae168a547d, 0x66952c92ecb40fc8, 0x2465cd79455e395b,
0x382145aada7578f, 0x7ad1a461044d611c, 0xbdc0865dfe733aa9, 0xff3067b657990c3a,
0x711223cfa3e5bb50, 0x33e2c2240a0f8dc3, 0xf4f3e018f031db76, 0xb60301f359dbe0e5,
0xda050215ea6c212f, 0x98f5e3fe438617bc, 0x5fe4c1c2b9b84c09, 0x1d14202910527a9a,
0x93366450e42ecdf0, 0xd1c685bb4dc4fb63, 0x16d7a787b7faa0d6, 0x5427466c1e109645,

```

0x4863ce9ff6e9f891, 0x0a932f745f03ce02, 0xcd820d48a53d95b7, 0x8f72eca30cd7a324,
0x0150a8daf8ab144e, 0x43a04931514122dd, 0x84b16b0dab7f7968, 0xc6418ae602954ffb,
0xbc387aea7a8da4c0, 0xfec89b01d3679253, 0x39d9b93d2959c9e6, 0x7b2958d680b3ff75,
0xf50b1caf74cf481f, 0xb7fbfd44dd257e8c, 0x70eadf78271b2539, 0x321a3e938ef113aa,
0x2e5eb66066087d7e, 0x6cae578bcfe24bed, 0xabbf75b735dc1058, 0xe94f945c9c3626cb,
0x676dd025684a91a1, 0x259d31cec1a0a732, 0xe28c13f23b9efc87, 0xa07cf2199274ca14,
0x167ff3eacba2af1, 0x548f120162451c62, 0x939e303d987b47d7, 0xd16ed1d631917144,
0x5f4c95afc5edc62e, 0x1dbc74446c07f0bd, 0xdaad56789639ab08, 0x985db7933fd39d9b,
0x84193f60d72af34f, 0xc6e9de8b7ec0c5dc, 0x01f8fcb784fe9e69, 0x43081d5c2d14a8fa,
0xcd2a5925d9681f90, 0x8fdab8ce70822903, 0x48cb9af28abc72b6, 0x0a3b7b1923564425,
0x70428b155b4eaf1e, 0x32b26afef2a4998d, 0xf5a348c2089ac238, 0xb753a929a170f4ab,
0x3971ed50550c43c1, 0x7b810cbbfce67552, 0xbc902e8706d82ee7, 0xfe60cf6caf321874,
0xe224479f47cb76a0, 0xa0d4a674ee214033, 0x67c58448141f1b86, 0x253565a3bdf52d15,
0xab1721da49899a7f, 0xe9e7c031e063acec, 0x2ef6e20d1a5df759, 0x6c0603e6b3b7c1ca,
0xf6fae5c07d3274cd, 0xb40a042bd4d8425e, 0x731b26172ee619eb, 0x31ebc7fc870c2f78,
0xbfc9838573709812, 0xfd39626eda9aae81, 0x3a28405220a4f534, 0x78d8a1b9894ec3a7,
0x649c294a61b7ad73, 0x266cc8a1c85d9be0, 0xe17dea9d3263c055, 0xa38d0b769b89f6c6,
0x2daf4f0f6ff541ac, 0x6f5faee4c61f773f, 0xa84e8cd83c212c8a, 0xeabe6d3395cb1a19,
0x90c79d3fedd3f122, 0xd2377cd44439c7b1, 0x15265ee8be079c04, 0x57d6bf0317edaa97,
0xd9f4fb7ae3911dfd, 0x9b041a914a7b2b6e, 0x5c1538adb04570db, 0x1ee5d94619af4648,
0x02a151b5f156289c, 0x4051b05e58bc1e0f, 0x87409262a28245ba, 0xc5b073890b687329,
0x4b9237f0ff14c443, 0x0962d61b56fef2d0, 0xce73f427acc0a965, 0x8c8315cc052a9ff6,
0x3a80143f5cf17f13, 0x7870f5d4f51b4980, 0xbf61d7e80f251235, 0xfd913603a6cf24a6,
0x73b3727a52b393cc, 0x31439391fb59a55f, 0xf652blad0167feea, 0xb4a25046a88dc879,
0xa8e6d8b54074a6ad, 0xea16395ee99e903e, 0x2d071b6213a0cb8b, 0x6ff7fa89ba4afd18,
0xel5bef04e364a72, 0xa3255f1be7dc7ce1, 0x64347d271de22754, 0x26c49cccb40811c7,
0x5cbd6cc0cc10fafc, 0x1e4d8d2b65facc6f, 0xd95caf179fc497da, 0x9bac4efc362ea149,
0x158e0a85c2521623, 0x577eeb6e6bb820b0, 0x906fc95291867b05, 0xd29f28b9386c4d96,
0xcdba04ad0952342, 0x8c2b41a1797f15d1, 0x4b3a639d83414e64, 0x09ca82762aab78f7,
0x87e8c60fdded7cf9d, 0xc51827e4773df90e, 0x020905d88d03a2bb, 0x40f9e43324e99428,
0x2cffe7d5975e55e2, 0x6e0f063e3eb46371, 0xa91e2402c48a38c4, 0xebeec5e96d600e57,
0x65cc8190991cb93d, 0x273c607b30f68fae, 0xe02d4247cac8d41b, 0xa2dda3ac6322e288,
0xbe992b5f8bdb8c5c, 0xfc69cab42231bacf, 0x3b78e888d80fe17a, 0x7988096371e5d7e9,
0xf7aa4d1a85996083, 0xb55aacf12c735610, 0x724b8ecd64d0da5, 0x30bb6f267fa73b36,
0x4ac29f2a07bfd00d, 0x08327ec1ae55e69e, 0xcf235cfd546bbd2b, 0x8dd3bd16fd818bb8,
0x03f1f96f09fd3cd2, 0x41011884a0170a41, 0x86103ab85a2951f4, 0xc4e0db53f3c36767,
0xd8a453a01b3a09b3, 0x9a54b24bb2d03f20, 0x5d45907748ee6495, 0x1fb5719ce1045206,
0x919735e51578e56c, 0xd367d40ebc92d3ff, 0x1476f63246ac884a, 0x568617d9ef46bed9,
0xe085162ab69d5e3c, 0xa275f7c11f7768af, 0x6564d5fde549331a, 0x279434164ca30589,
0xa9b6706fb8dfb2e3, 0xeb46918411358470, 0x2c57b3b8eb0bdfc5, 0x6ea7525342e1e956,
0x72e3daa0aa188782, 0x30133b4b03f2b111, 0xf7021977f9ccea4, 0xb5f2f89c5026dc37,
0x3bd0bce5a45a6b5d, 0x79205d0e0db05dce, 0xbe317f32f78e067b, 0xfcc19ed95e6430e8,
0x86b86ed5267cddb3, 0xc4488f3e8f96ed40, 0x0359ad0275a8b6f5, 0x41a94ce9dc428066,
0xcf8b0890283e370c, 0x8d7be97b81d4019f, 0x4a6acb477bea5a2a, 0x089a2aacd2006cb9,
0x14dea25f3af9026d, 0x562e43b4931334fe, 0x913f6188692d6f4b, 0xd3cf8063c0c759d8,

```
0x5dedc41a34bbeeb2, 0x1f1d25f19d51d821, 0xd80c07cd676f8394, 0x9afce626ce85b507
```

5.12.2 Mirrored CRC

The CRC for a single byte is calculated as follows:

```
UInt8 CalculateCrcFor1Byte(UInt8 data, UInt64 crc) {
    return crcTable[(crc ^ data) & 0xff] ^ (crc >> 8);
}
```

The CRC table is initialized with the following values:

```
0x0000000000000000, 0x7ad870c830358979, 0xf5b0e190606b12f2, 0x8f689158505e9b8b,
0xc038e5739841b68f, 0xbae095bba8743ff6, 0x358804e3f82aa47d, 0x4f50742bc81f2d04,
0xab28ecb46814fe75, 0xd1f09c7c5821770c, 0x5e980d24087fec87, 0x24407dec384a65fe,
0x6b1009c7f05548fa, 0x11c8790fc060c183, 0x9ea0e857903e5a08, 0xe478989fa00bd371,
0x7d08ff3b88be6f81, 0x07d08ff3b88be6f8, 0x88b81eabe8d57d73, 0xf2606e63d8e0f40a,
0xbd301a4810ffd90e, 0xc7e86a8020ca5077, 0x4880fbd87094cbfc, 0x32588b1040a14285,
0xd620138fe0aa91f4, 0xacf86347d09f188d, 0x2390f21f80c18306, 0x594882d7b0f40a7f,
0x1618f6fc78eb277b, 0x6cc0863448deae02, 0xe3a8176c18803589, 0x997067a428b5bcf0,
0xfa11fe77117cdf02, 0x80c98ebf2149567b, 0x0fa11fe77117cdf0, 0x75796f2f41224489,
0x3a291b04893d698d, 0x40f16bccb908e0f4, 0xcf99fa94e9567b7f, 0xb5418a5cd963f206,
0x513912c379682177, 0x2be1620b495da80e, 0xa489f35319033385, 0xde51839b2936bafc,
0x9101f7b0e12997f8, 0xebd98778d11c1e81, 0x64b116208142850a, 0x1e6966e8b1770c73,
0x8719014c99c2b083, 0xfdc17184a9f739fa, 0x72a9e0dcf9a9a271, 0x08719014c99c2b08,
0x4721e43f0183060c, 0x3df994f731b68f75, 0xb29105af61e814fe, 0xc849756751dd9d87,
0x2c31edf8f1d64ef6, 0x56e99d30c1e3c78f, 0xd9810c6891bd5c04, 0xa3597ca0a188d57d,
0xec09088b6997f879, 0x96d1784359a27100, 0x19b9e91b09fcea8b, 0x636199d339c963f2,
0xdf7adabd7a6e2d6f, 0xa5a2aa754a5ba416, 0x2aca3b2d1a053f9d, 0x50124be52a30b6e4,
0x1f423fcee22f9be0, 0x659a4f06d21a1299, 0xeaf2de5e82448912, 0x902aae96b271006b,
0x74523609127ad31a, 0x0e8a46c1224f5a63, 0x81e2d7997211c1e8, 0xfb3aa75142244891,
0xb46ad37a8a3b6595, 0xceb2a3b2ba0eecec, 0x41da32eaea507767, 0x3b024222da65fe1e,
0xa2722586f2d042ee, 0xd8aa554ec2e5cb97, 0x57c2c41692bb501c, 0x2d1ab4dea28ed965,
0x624ac0f56a91f461, 0x1892b03d5aa47d18, 0x97fa21650afae693, 0xed2251ad3acf6fea,
0x095ac9329ac4bc9b, 0x7382b9faaf135e2, 0xfcea28a2faafae69, 0x8632586aca9a2710,
0xc9622c4102850a14, 0xb3ba5c8932b0836d, 0x3cd2cdd162ee18e6, 0x460abd1952db919f,
0x256b24ca6b12f26d, 0x5fb354025b277b14, 0xd0dbc55a0b79e09f, 0xaa03b5923b4c69e6,
0xe553c1b9f35344e2, 0x9f8bb171c366cd9b, 0x10e3202993385610, 0x6a3b50e1a30ddf69,
0x8e43c87e03060c18, 0xf49bb8b633338561, 0x7bf329ee636d1eea, 0x012b592653589793,
0x4e7b2d0d9b47ba97, 0x34a35dc5ab7233ee, 0xbcbcc9dfb2ca865, 0xc113bc55cb19211c,
0x5863dbf1e3ac9dec, 0x22bbab39d3991495, 0xadd33a6183c78f1e, 0xd70b4aa9b3f20667,
0x985b3e827bed2b63, 0xe2834e4a4bd8a21a, 0x6debdf121b863991, 0x1733afda2bb3b0e8,
0xf34b37458bb86399, 0x8993478dbb8deae0, 0x06fbd6d5ebd3716b, 0x7c23a61ddbe6f812,
```

```

0x3373d23613f9d516, 0x49aba2fe23cc5c6f, 0xc6c333a67392c7e4, 0xbc1b436e43a74e9d,
0x95ac9329ac4bc9b5, 0xef74e3e19c7e40cc, 0x601c72b9cc20db47, 0x1ac40271fc15523e,
0x5594765a340a7f3a, 0x2f4c0692043ff643, 0xa02497ca54616dc8, 0xdafce7026454e4b1,
0x3e847f9dc45f37c0, 0x445c0f55f46abeb9, 0xcb349e0da4342532, 0xb1ecec59401ac4b,
0xfebc9aee5c1e814f, 0x8464ea266c2b0836, 0x0b0c7b7e3c7593bd, 0x71d40bb60c401ac4,
0xe8a46c1224f5a634, 0x927c1cda14c02f4d, 0x1d148d82449eb4c6, 0x67ccfd4a74ab3dbf,
0x289c8961bcb410bb, 0x5244f9a98c8199c2, 0xdd2c68f1dcdf0249, 0xa7f41839ecea8b30,
0x438c80a64ce15841, 0x3954f06e7cd4d138, 0xb63c61362c8a4ab3, 0xcce411fe1cbfc3ca,
0x83b465d5d4a0eece, 0xf96c151de49567b7, 0x76048445b4cbfc3c, 0x0cdcf48d84fe7545,
0x6fbd6d5ebd3716b7, 0x15651d968d029fce, 0x9a0d8ccedd5c0445, 0xe0d5fc06ed698d3c,
0xaf85882d2576a038, 0xd55df8e515432941, 0x5a3569bd451db2ca, 0x20ed197575283bb3,
0xc49581ead523e8c2, 0xbe4df122e51661bb, 0x3125607ab548fa30, 0x4bfd10b2857d7349,
0x04ad64994d625e4d, 0x7e7514517d57d734, 0xf11d85092d094cbf, 0x8bc5f5c11d3cc5c6,
0x12b5926535897936, 0x686de2ad05bcf04f, 0xe70573f555e26bc4, 0x9ddd033d65d7e2bd,
0xd28d7716adc8cfb9, 0xa85507de9dfd46c0, 0x273d9686cda3dd4b, 0x5de5e64efd965432,
0xb99d7ed15d9d8743, 0xc3450e196da80e3a, 0x4c2d9f413df695b1, 0x36f5ef890dc31cc8,
0x79a59ba2c5dc31cc, 0x037deb6af5e9b8b5, 0x8c157a32a5b7233e, 0xf6cd0afa9582aa47,
0x4ad64994d625e4da, 0x300e395ce6106da3, 0xbf66a804b64ef628, 0xc5bed8cc867b7f51,
0x8aeeace74e645255, 0xf036dc2f7e51db2c, 0x7f5e4d772e0f40a7, 0x05863dbf1e3ac9de,
0xe1fea520be311aaf, 0x9b26d5e88e0493d6, 0x144e44b0de5a085d, 0x6e963478ee6f8124,
0x21c640532670ac20, 0x5b1e309b16452559, 0xd476a1c3461bbbed2, 0xaaead10b762e37ab,
0x37deb6af5e9b8b5b, 0x4d06c6676eae0222, 0xc26e573f3ef099a9, 0xb8b627f70ec510d0,
0xf7e653dcc6da3dd4, 0x8d3e2314f6efb4ad, 0x0256b24ca6b12f26, 0x788ec2849684a65f,
0x9cf65a1b368f752e, 0xe62e2ad306bafc57, 0x6946bb8b56e467dc, 0x139ecb4366d1eea5,
0x5ccebfb68aeecc3a1, 0x2616cfa09efb4ad8, 0xa97e5ef8cea5d153, 0xd3a62e30fe90582a,
0xb0c7b7e3c7593bd8, 0xca1fc72bf76cb2a1, 0x45775673a732292a, 0x3faf26bb9707a053,
0x70ff52905f188d57, 0x0a2722586f2d042e, 0x854fb3003f739fa5, 0xff97c3c80f4616dc,
0x1bef5b57af4dc5ad, 0x61372b9f9f784cd4, 0xee5fbac7cf26d75f, 0x9487ca0fff135e26,
0xdbd7be24370c7322, 0xa10fceed0739fa5b, 0x2e675fb4576761d0, 0x54bf2f7c6752e8a9,
0xcdcf48d84fe75459, 0xb71738107fd2dd20, 0x387fa9482f8c46ab, 0x42a7d9801fb9cfd2,
0x0df7adabd7a6e2d6, 0x772fdd63e7936baf, 0xf8474c3bb7cdf024, 0x829f3cf387f8795d,
0x66e7a46c27f3aa2c, 0x1c3fd4a417c62355, 0x935745fc4798b8de, 0xe98f353477ad31a7,
0xa6df411fbfb21ca3, 0xdc0731d78f8795da, 0x536fa08fd90e51, 0x29b7d047efec8728

```

5.13 Reed-Solomon encoding

R21 uses Reed-Solomon (RS) encoding to add error correction. Error correction codes are typically used in telecommunication to correct errors during transmission or on media to correct e.g. errors caused by a scratch on a CD. RS coding takes considerably study to master, and books on the subject require at least some mathematical base knowledge on academic level. For this reason it's recommended to use an existing RS implementation, rather than to build one from scratch. When choosing to learn about the subject, a good book on the subject is "Error Control Coding, Second Edition", by Shu Lin and Daniel J. Costello, Jr. This book is taught over two semesters, to give an idea of the depth of the subject. RS coding is treated in Chapter 7 out of 22, to have a full understanding of the subject chapters 1-7 should be read.

An open source RS implementation is available from <http://www.eccpage.com/>, item “Reed-Solomon (RS) codes”, by Simon Rockliff, 1989. This implementation uses Berlekamp-Massey for decoding. Note that there are many ways to encode and decode, the implementation above is just one example. Though only 404 lines of code, the math involved is very sophisticated.

DWG file format version R21 uses two configurations of RS coding:

- Data pages: use a (n, k) of (255, 251), the primitive polynomial coefficients being (1, 0, 1, 1, 1, 0, 0, 0). This configuration can correct $(255 - 251) / 2 = 2$ error bytes per block of 255 bytes. For each 251 data bytes (k), 4 parity bytes are added to form a 255 byte (code word) block.
- System pages: use a (n, k) of (255, 239), the primitive polynomial coefficients being (1, 0, 0, 1, 0, 1, 1, 0). This configuration can correct $(255 - 239) / 2 = 8$ error bytes per block of 255 bytes. For each 239 data bytes (k), 16 parity bytes are added to form a 255 byte (code word) block.

In the RS implementation by Simon Rockliff the primitive polynomial coefficients are stored in variable pp. From these coefficients the lookup tables for Galois field math and the generator polynomial coefficients are created.

The encoded bytes may be interleaved depending on the context. Some data/system pages are interleaved, some are not.

5.13.1 Non-interleaved

All original data blocks are followed by the parity byte blocks (i.e. the first parity block follows the last data block).

When the last block is not entirely filled, then random bytes are added from the random encoding (see paragraph 5.11) to fill the block to have size k.

5.13.2 Interleaved

When more than 1 block of data is encoded, the encoded block data is interleaved. E.g. when there are 3 blocks to be encoded, then the data bytes and parity bytes of the first block are written to positions $3 \times i$ (where i is an integer ≥ 0). The encoded bytes of the second block are written to positions $3 \times i + 1$ and of the third block to positions $3 \times i + 2$.

When the last block is not entirely filled, then random bytes are added from the random encoding (see paragraph 5.11) to fill the block to have size k.